

TruncateDiv 算子设计方案

1. 背景介绍

1.1 需求来源

TruncateDiv 是 Ascend C 内置算子，实现逐元素除法后向零取整功能。本方案使用 Ascend C 重新实现该算子，支持 BF16/FP16/FP32 三种数据类型，并在 ascend910b/ascend950 上达到与内置算子一致的功能和性能。

1.2 数学定义

$$y = \text{trunc} \left(\frac{x1}{x2} \right)$$

其中 trunc 为向零取整操作（舍去小数部分）， $x1$ 、 $x2$ 为支持广播的输入张量。

1.3 内置算子参考

内置算子实现路

径： /usr/local/Ascend/ascend-toolkit/latest/opp/built-in/op_impl/ai_core/tbe/impl

2. 外部资源

2.1 涉及外部依赖

依赖	说明
CANN Toolkit 9.0.0	Ascend C 编译工具链和运行时库
kernel_operator.h	Ascend C Kernel 算子框架头文件
register/op_def_registry.h	算子定义注册

依赖	说明
register/op_impl_registry.h	算子实现注册（InferShape 等）
ascendc/host_api/tiling/	Tiling 模板参数定义

2.2 内部接口模块

- **OpDef 接口**：算子定义注册，声明输入/输出/属性
- **InferShape 接口**：输出形状推导（广播规则）
- **TilingFunc 接口**：计算 tiling 数据和分核策略
- **Kernel 入口**：模板化核函数，通过 TilingKey 分派不同数据类型路径

3. 功能设计

3.1 算子原型

参数	方向	dtype	format	shape	说明
x1	输入	fp16/fp32/bf16	ND	all	被除数
x2	输入	fp16/fp32/bf16	ND	all	除数
y	输出	fp16/fp32/bf16	ND	同广播结果	结果：trunc(x1/x2)

3.2 属性检查

- 输入 x1、x2 必须具有相同的数据类型
- 输入维度数 ≤ 8 (kMaxDim)
- x1 和 x2 的形状必须满足广播兼容规则

3.3 算子实现约束

- **不支持 bfloat16 类型直接使用 Div API**：Ascend 910b Vector/Memory 层 Div 仅支持 half 和 float。bfloat16_t 必须通过 Cast→fp32→计算→Cast→bf16 路径实现。
- **DataCopy 的 32B 对齐约束**：DataCopy API 要求源/目的地址和搬运大小严格 32 字节对齐。非对齐场景必须使用 DataCopyPad。
- **Trunc 高阶 API 需要临时缓冲区**：通过 sharedTmpBuffer 或框架管理方式提供。

3.4 功能详细设计

3.4.1 Host 侧设计

Tiling 策略：

1. **广播形状计算：**通过 `GetInputShape` 获取 `x1`、`x2` 的原始形状，维度右对齐后逐维广播（`dim11` 或 `dim21` 或 `dim1dim2`），计算输出形状和总元素数 `totalNum`。标量输入（`GetDimNum0`）视为 `shape={1}`，可广播到任意维度。
2. **分核策略：**
 - `totalNum ≤ MIN_SPLIT_THRESHOLD(1024)`：单核处理（`blockDim=1`）
 - `totalNum > MIN_SPLIT_THRESHOLD`：多核处理，核数 = `min(coreNum, ceil(totalNum / MIN_ELEMS_PER_CORE))`
 - `MIN_ELEMS_PER_CORE = 2048`：保证每核至少处理 2048 个元素，避免核数过多导致每核开销占比过大
 - 多核场景下 `perCoreNum = CeilAlign(perCoreNum, alignElems)`，确保各核 GM 偏移为 32 字节对齐
 - 使用 `CeilAlign`（向上对齐）而非 `FloorAlign`，避免 `perCoreNum × blockDim < totalNum` 导致末尾元素丢失
3. **UB 内存分配（CalculateUbFactor）：**
 - 使用 TQue 双缓冲：`x1(2) + x2(2) + y(2) = 6` 个 slot
 - `fp16/bf16 Cast` 路径：`TQue (6×2=12 bytes/elem) + fp32 TBuf (3×4=12 bytes/elem) = 24 bytes/elem`
 - `fp32 直接路径`：`TQue (6×4=24 bytes/elem) = 24 bytes/elem`
 - 两轮计算：第一轮忽略 `tmpSize` 算 `ubFactor` → 根据 `ubFactor` 算 `tmpSize` → 第二轮扣除 `tmpSize` 后重新算 `ubFactor`
 - `Trunc 临时缓冲区`：`tmpSize = ubFactor × sizeof(float) + 256`
4. **TilingKey 规划：**

TilingKey	值	数据类型	说明
MODE_0	0	fp16	使用 Cast→fp32 路径计算
MODE_1	1	fp32	直接路径计算
MODE_2	2	bf16	使用 Cast→fp32 路径计算

TilingKey 使用 2-bit 位宽编码，通过 `ASCENDC_TPL_UINT_DECL(schMode, 2, ...)` 声明。

5. **TilingData 数据结构：**

```

struct TruncateDivTilingData {
    int64_t totalNum;        // 广播后总元素数量
    int64_t blockFactor;    // 每个核处理的元素数量
    int64_t ubFactor;       // 每次 UB 循环处理的元素数量
    int64_t x1ElemNum;      // x1 原始元素数（广播前，用于判断是否需要广播路径）
    int64_t x2ElemNum;      // x2 原始元素数
    int64_t dimNum;         // 广播后维度数（0=标量）
    int64_t outShape[8];    // 广播后输出形状
    int64_t x1Shape[8];     // x1 形状（已补1对齐）
    int64_t x2Shape[8];     // x2 形状（已补1对齐）
    int64_t tmpSize;        // Trunc 临时缓冲区大小（字节）
};

```

分核策略：

- 同 shape（非广播）：使用快速路径，TQue 双缓冲流水线，CopyIn 与 Compute/CopyOut 重叠
- 广播 shape：使用广播路径，TBuf + SetFlag/WaitFlag 同步，分块 gather→compute→scatter

UB 使用原则：

- UB 预留 4096 字节（UB_RESERVE_SIZE）用于栈/框架开销
- TQue 双缓冲（2 slot）实现 CopyIn 与 Compute 的重叠流水
- bf16/fp16 路径额外分配 fp32 中间缓冲区用于高精度计算
- Trunc 高阶 API 的临时缓冲区统一用 sharedTmpBuffer 方式管理

3.4.2 Kernel 侧设计

总体架构：

```

template <uint32_t schMode>
__global__ __aicore__ void truncate_div(GM_ADDR x1, GM_ADDR x2, GM_ADDR y,
                                         GM_ADDR workspace, GM_ADDR tiling)

```

通过模板参数 schMode（0/1/2）区分数据类型，编译期即确定计算路径，避免运行时分支开销。

Kernel 类（TruncateDiv）：

方法	说明
Init	解析 TilingData，初始化 GM Tensor 引用和 UB 缓冲区
Process	入口方法，路由到快速路径或广播路径

方法	说明
CopyInFast	DataCopyPad 从 GM 搬运 x1+x2 到 UB TQue
ComputeFast	执行 Div + Trunc（Cast 路径额外含 Cast 操作）
CopyOutFast	DataCopyPad 从 UB TQue 搬运结果到 GM
ProcessBroadcast	广播路径：分块 gather→compute→scatter
MapIndex	将输出 flat index 映射为广播后的输入 flat index

快速路径流水线：

```
CopyIn(Tile1) | ---
CopyIn(Tile2) | Compute(Tile1) + CopyOut(Tile1)
CopyIn(Tile3) | Compute(Tile2) + CopyOut(Tile2)
...          | ...
              | Compute>LastTile) + CopyOut>LastTile)
```

通过 TQue<2> 双缓冲实现 CopyIn 与 Compute/CopyOut 的重叠。

bf16/fp16 Cast 路径（kUseCast = true）：

- 1. DeQue bf16/fp16 数据
- 2. Cast<float, T>(fp32Buf, bf16Buf, CAST_NONE, N) — 低精度→fp32
- 3. Div(fp32Buf_y, fp32Buf_x1, fp32Buf_x2, N) — fp32 除法
- 4. Trunc<float>(fp32Buf_y, fp32Buf_y, tmpBuf, N) — fp32 截断
- 5. Cast<T, float>(bf16Buf_y, fp32Buf_y, CAST_ROUND, N) — fp32→低精度
- 6. EnQue 结果

fp32 直接路径（kUseCast = false）：

- 1. DeQue fp32 数据
- 2. Div(y, x1, x2, N) — fp32 除法
- 3. Trunc<float>(y, y, tmpBuf, N) — fp32 截断
- 4. EnQue 结果

广播路径：

- Block Size：256 元素/块
- Gather：按广播规则通过 MapIndex 计算输入索引，逐元素 DataCopyPad 搬运
- 同步：SetFlag<MTE2_V> / WaitFlag<MTE2_V> 确保 DMA 完成后计算

- 同步： SetFlag<V_MTE3> / WaitFlag<V_MTE3> 确保计算完成后 Scatter
- Scatter：逐元素 DataCopyPad 写回输出
- Cast 路径同样支持（bf16/fp16 广播使用 fp32 中间计算）

3.5 使能方式

框架	支持
TF 训练/推理	—
PyTorch 训练/推理	—
ATC 转换	—
Aclnn 直调	✓
OPAT 调测	—
SGAT 图编译	—

4. 支持硬件

芯片	支持状态
ascend910b	✓
ascend950	✓

5. 约束与限制

#	约束
1	输入 x1、x2 数据类型必须一致
2	最大维度数 ≤ 8（kMaxDim）
3	bfloat16 不支持直接使用 Div API，必须通过 Cast→fp32 路径
4	GM↔UB 搬运非 32B 对齐时，必须使用 DataCopyPad 替代 DataCopy

#	约束
5	Trunc 高阶 API 需要临时缓冲区，大小通过公式 $\text{ubFactor} \times \text{sizeof(float)} + 256$ 估算
6	不支持输入包含 NaN/Inf 时的特殊行为保证

6. 测试建议

6.1 功能测试

测试维度	覆盖范围
数据类型	fp16, fp32, bf16
Shape 组合	同 shape、广播（标量广播、一维广播、多维广播、右对齐广播）
边界值	除数为 0、全 0 输入、极大/极小值、负数除法后截断
分核边界	$\text{totalNum} \leq 1024$ （单核）、 $\text{totalNum} > 1024$ （多核）

6.2 精度标准

数据类型	阈值	说明
float16	$2^{-10} \approx 0.000977$	$\text{MARE} < 10 \times \text{阈值}$
float32	$2^{-13} \approx 0.000122$	$\text{MARE} < 10 \times \text{阈值}$
bfloat16	$2^{-7} \approx 0.00781$	$\text{MARE} < 10 \times \text{阈值}$

6.3 性能标准

- 对比对象：TBE 内置 TruncateDiv 算子
- 评测平台：CANNJudge（50 个测试用例，覆盖所有 dtype 和 shape 组合）
- 目标：综合得分 ≥ 90

7. 附录

7.1 文件结构

```
code/
├── op_host/
│   ├── truncate_div_def.cpp          # 算子定义
│   ├── truncate_div_infershape.cpp  # 形状推导
│   └── truncate_div_tiling.cpp      # Tiling 实现
└── op_kernel/
    ├── truncate_div.h                # Kernel 类定义与实现
    ├── truncate_div.cpp              # Kernel 入口
    ├── truncate_div_tiling_key.h     # TilingKey 定义
    └── truncate_div_tiling_data.h    # TilingData 结构
```

7.2 关键设计决策

决策	原因
fp16 也使用 Cast→fp32 路径	避免 Div 结果在整数边界处因 fp16 精度不足被错误舍入
CeilAlign 而非 FloorAlign	确保 $perCoreNum \times blockDim \geq totalNum$ ，避免丢失末尾元素
MIN_ELEMS_PER_CORE=2048	平衡多核并行度与每核算子初始化开销，避免 64 核各处理 ~80 元素的低效场景
不使用 GetTruncMaxMinTmpSize()	CANN 9.0.0 下该 API 头文件 include path 不兼容，改用公式直接计算
DataCopyPad 全程使用	无法在编译期确定数据对齐性，DataCopyPad 兼容对齐和非对齐场景